Week 5
02-03-2026
Penguins Business Intelligence Dashboard Requirement.

Build a **complete interactive dashboard using**:

- Dash
- Plotly
- pandas
- Seaborn (for dataset)

Dataset used: **Palmer Penguins dataset**

STEP 1 — Install Required Libraries

In VS Code terminal:

```
pip install dash plotly pandas seaborn numpy
```

STEP 2 — Dashboard Code

Create file: `penguins_dashboard.py`

```
import pandas as pd
import numpy as np
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go
from dash import Dash, html, dcc, Input, Output

# =========================
# LOAD DATA
# =========================

df = sns.load_dataset("penguins")

# Data Cleaning
df = df.dropna()

# =========================
# DASH APP
# =========================

app = Dash(__name__)

# =========================
# LAYOUT
# =========================

app.layout = html.Div([

    html.H1("Penguins Executive Dashboard", style={"textAlign": "center"}),
```

```
    # ========================
    # FILTERS
    # ========================

    html.Div([
        dcc.Dropdown(
            id="species_filter",
            options=[{"label": s, "value": s} for s in
df["species"].unique()],
            multi=True,
            placeholder="Select Species"
        ),

        dcc.Dropdown(
            id="island_filter",
            options=[{"label": i, "value": i} for i in
df["island"].unique()],
            multi=True,
            placeholder="Select Island"
        ),

        dcc.Dropdown(
            id="gender_filter",
            options=[{"label": g, "value": g} for g in df["sex"].unique()],
            multi=True,
            placeholder="Select Gender"
        )

    ], style={"display": "grid",
              "gridTemplateColumns": "repeat(3, 1fr)",
              "gap": "10px",
              "padding": "20px"}),

    # ========================
    # KPI CARDS
    # ========================

    html.Div(id="kpi_cards",
             style={"display": "grid",
                    "gridTemplateColumns": "repeat(5, 1fr)",
                    "gap": "15px",
                    "padding": "20px"}),

    # ========================
    # CHARTS
    # ========================

    dcc.Graph(id="species_chart"),
    dcc.Graph(id="bodymass_chart"),
    dcc.Graph(id="scatter_chart"),
    dcc.Graph(id="correlation_chart")

])

# ========================
# CALLBACK
# ========================

@app.callback(
    Output("kpi_cards", "children"),
```

```
    Output("species_chart", "figure"),
    Output("bodymass_chart", "figure"),
    Output("scatter_chart", "figure"),
    Output("correlation_chart", "figure"),
    Input("species_filter", "value"),
    Input("island_filter", "value"),
    Input("gender_filter", "value")
)
def update_dashboard(species, island, gender):

    dff = df.copy()

    if species:
        dff = dff[dff["species"].isin(species)]
    if island:
        dff = dff[dff["island"].isin(island)]
    if gender:
        dff = dff[dff["sex"].isin(gender)]

    # ================= KPI CALCULATIONS =================

    total_penguins = len(dff)
    avg_body_mass = dff["body_mass_g"].mean()
    avg_flipper = dff["flipper_length_mm"].mean()
    avg_bill = dff["bill_length_mm"].mean()

    male_pct = (len(dff[dff["sex"] == "Male"]) / total_penguins * 100) if
total_penguins else 0
    female_pct = (len(dff[dff["sex"] == "Female"]) / total_penguins * 100)
if total_penguins else 0

    card_style = {
        "backgroundColor": "#f4f4f4",
        "padding": "15px",
        "borderRadius": "10px",
        "textAlign": "center",
        "boxShadow": "2px 2px 8px rgba(0,0,0,0.1)"
    }

    kpis = [
        html.Div([html.H4("Total Penguins"),
                  html.H2(total_penguins)], style=card_style),

        html.Div([html.H4("Avg Body Mass (g)"),
                  html.H2(f"{avg_body_mass:.2f}")], style=card_style),

        html.Div([html.H4("Avg Flipper Length (mm)"),
                  html.H2(f"{avg_flipper:.2f}")], style=card_style),

        html.Div([html.H4("Avg Bill Length (mm)"),
                  html.H2(f"{avg_bill:.2f}")], style=card_style),

        html.Div([html.H4("Male vs Female %"),
                  html.H2(f"{male_pct:.1f}% / {female_pct:.1f}%")],
style=card_style)
    ]

    # ================= CHARTS =================

    species_chart = px.bar(dff, x="species", title="Species Distribution")
```

```
    bodymass_chart = px.bar(
        dff.groupby("species")["body_mass_g"].mean().reset_index(),
        x="species", y="body_mass_g",
        title="Average Body Mass by Species"
    )

    scatter_chart = px.scatter(
        dff,
        x="flipper_length_mm",
        y="body_mass_g",
        color="species",
        size="bill_length_mm",
        title="Flipper Length vs Body Mass"
    )

    corr = dff[["body_mass_g", "flipper_length_mm",
                "bill_length_mm", "bill_depth_mm"]].corr()

    correlation_chart = px.imshow(
        corr,
        text_auto=True,
        title="Correlation Heatmap"
    )

    return kpis, species_chart, bodymass_chart, scatter_chart,
correlation_chart


# ========================
# RUN APP
# ========================

if __name__ == "__main__":
    app.run(debug=True)
```

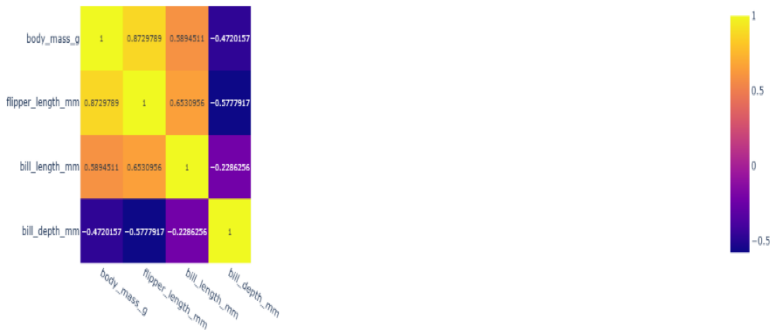
STEP 3 — Run in VS Code

```
python penguins_dashboard.py
```

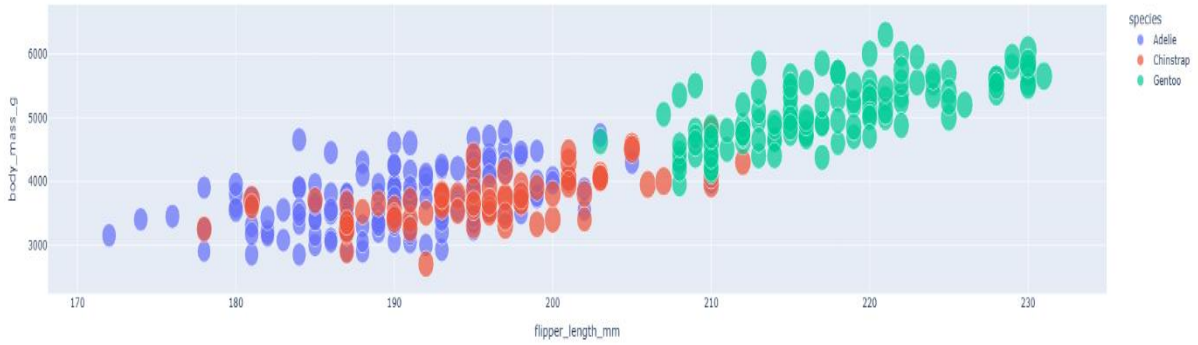
Open:

```
http://127.0.0.1:8050/
```

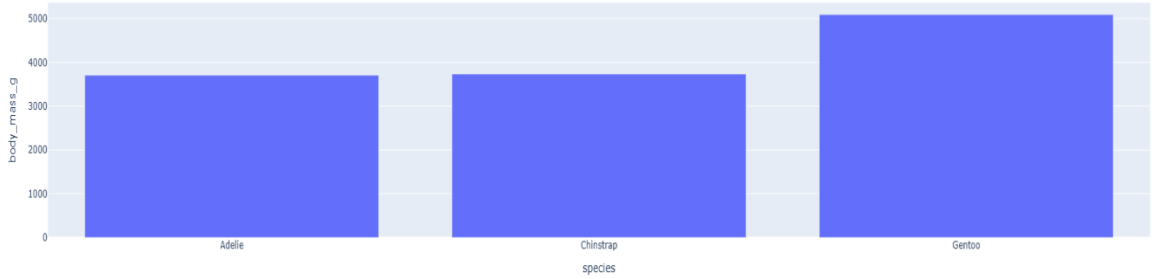
Correlation Heatmap



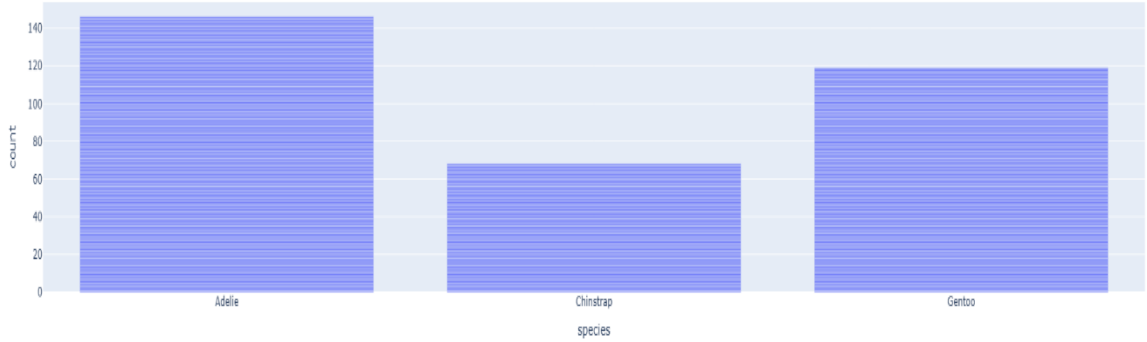
Flipper Length vs Body Mass



Average Body Mass by Species



Species Distribution



03-03-2026

Professional executive-level Auto MPG dashboard project.

Build it using:

- Dash
- Plotly
- pandas
- Seaborn

Dataset: **Auto MPG Dataset**

STEP 1 — Install Required Libraries

In VS Code terminal:

```
pip install dash plotly pandas seaborn numpy
```

STEP 2 — Create File

Create:

```
auto_mpg_dashboard.py
```

STEP 3 —DASHBOARD CODE

```
import pandas as pd
import numpy as np
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go
from dash import Dash, html, dcc

# =========================
# LOAD DATA
# =========================

df = sns.load_dataset("mpg")

# Data Cleaning
df = df.dropna()

# Rename columns for clarity
df.rename(columns={
    "origin": "Origin",
    "model_year": "Model Year",
    "horsepower": "Horsepower",
    "weight": "Weight",
    "mpg": "MPG",
    "displacement": "Displacement",
    "cylinders": "Cylinders",
```

```
    "acceleration": "Acceleration"
}, inplace=True)

# ========================
# KPI CALCULATIONS
# ========================

total_vehicles = len(df)
avg_mpg = df["MPG"].mean()
max_mpg = df["MPG"].max()
min_mpg = df["MPG"].min()
avg_hp = df["Horsepower"].mean()

# MPG YoY Growth
mpg_by_year = df.groupby("Model Year")["MPG"].mean().reset_index()
yoy_growth = ((mpg_by_year["MPG"].iloc[-1] - mpg_by_year["MPG"].iloc[0])
              / mpg_by_year["MPG"].iloc[0]) * 100

# Correlations
corr_matrix = df[["MPG", "Weight", "Horsepower",
                  "Displacement", "Cylinders",
                  "Acceleration"]].corr()

mpg_weight_corr = corr_matrix.loc["MPG", "Weight"]
mpg_hp_corr = corr_matrix.loc["MPG", "Horsepower"]

# ========================
# DASH APP
# ========================

app = Dash(__name__)

card_style = {
    "backgroundColor": "#f2f2f2",
    "padding": "15px",
    "borderRadius": "10px",
    "textAlign": "center",
    "boxShadow": "2px 2px 8px rgba(0,0,0,0.1)"
}

app.layout = html.Div([

    html.H1("Auto MPG Executive Dashboard",
            style={"textAlign": "center"}),

    # ================= KPI CARDS =================
    html.Div([
        html.Div([html.H4("Average MPG"),
                  html.H2(f"{avg_mpg:.2f}")], style=card_style),

        html.Div([html.H4("MPG YoY Growth %"),
                  html.H2(f"{yoy_growth:.2f}%")], style=card_style),

        html.Div([html.H4("Average Horsepower"),
                  html.H2(f"{avg_hp:.2f}")], style=card_style),

        html.Div([html.H4("Total Vehicles"),
                  html.H2(total_vehicles)], style=card_style)

    ], style={"display": "grid",
              "gridTemplateColumns": "repeat(4, 1fr)",
```

```
            "gap": "15px",
            "padding": "20px"}),

# ================= MIDDLE SECTION =================

dcc.Graph(
    figure=px.line(
        mpg_by_year,
        x="Model Year",
        y="MPG",
        title="Fuel Efficiency Trend (MPG Over Years)",
        markers=True
    )
),

dcc.Graph(
    figure=px.bar(
        df.groupby("Origin")["MPG"].mean().reset_index(),
        x="Origin",
        y="MPG",
        title="Average MPG by Origin"
    )
),

# ================= BOTTOM SECTION =================

dcc.Graph(
    figure=px.scatter(
        df,
        x="Weight",
        y="MPG",
        title=f"MPG vs Weight (Corr: {mpg_weight_corr:.2f})"
    )
),

dcc.Graph(
    figure=px.scatter(
        df,
        x="Horsepower",
        y="MPG",
        title=f"MPG vs Horsepower (Corr: {mpg_hp_corr:.2f})"
    )
),

dcc.Graph(
    figure=px.histogram(
        df,
        x="MPG",
        nbins=30,
        title="MPG Distribution"
    )
),

dcc.Graph(
    figure=px.imshow(
        corr_matrix,
        text_auto=True,
        title="Correlation Matrix"
    )
)
```

```
])

# ========================
# RUN APP
# ========================

if __name__ == "__main__":
    app.run(debug=True)
```

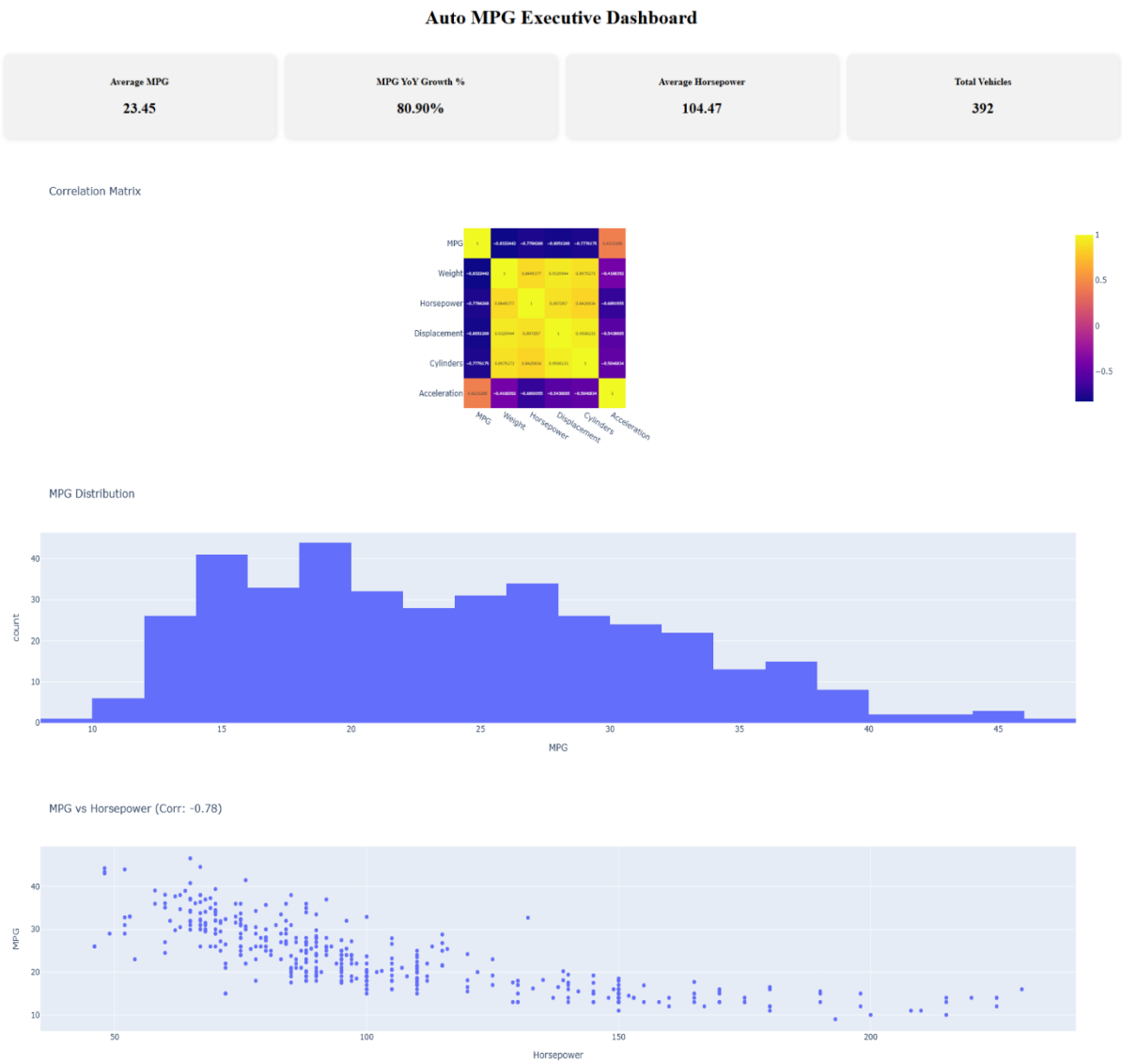
STEP 4 — Run in VS Code

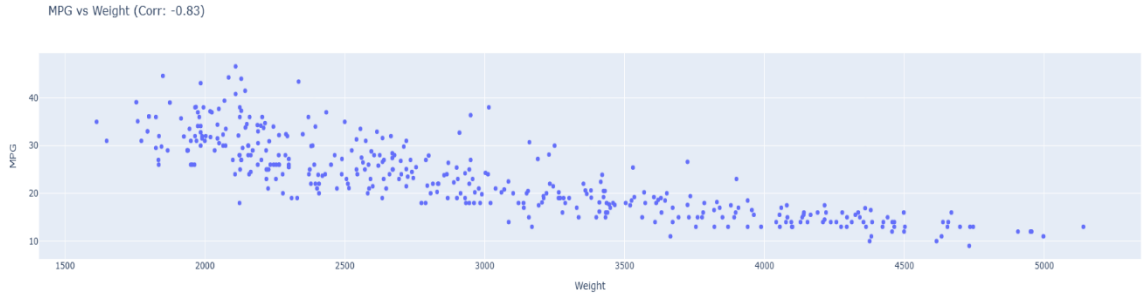
Open terminal inside folder and run:

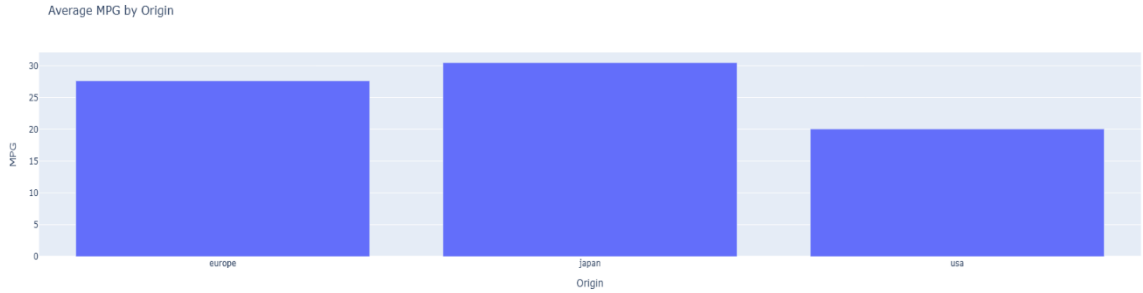
```
python auto_mpg_dashboard.py
```

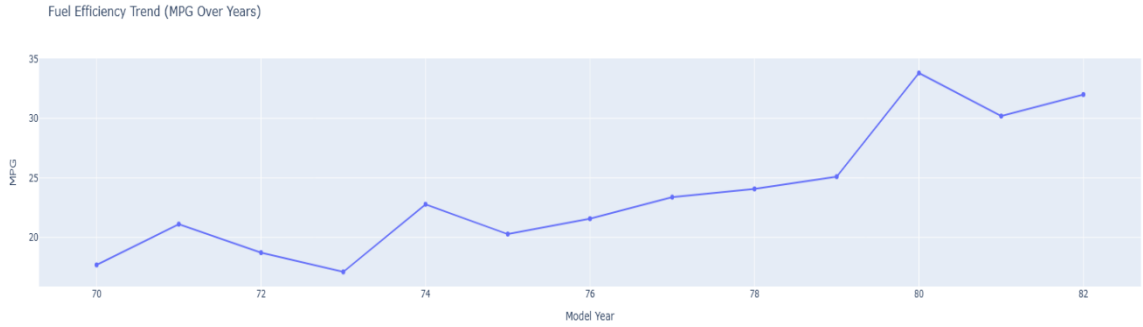
Open browser:

http://127.0.0.1:8050/









04-03-2026

SQL & DBMS – Assignment Answers

1. What is SQL?

SQL (Structured Query Language) is a standard programming language used to manage and manipulate relational databases. It is used to create, retrieve, update, and delete data stored in database systems such as MySQL, PostgreSQL, Oracle, and SQL Server. SQL allows users to interact with databases efficiently using structured queries.

2. Difference Between DDL, DML, and DCL

Category	Full Form	Purpose	Examples
DDL	Data Definition Language	Defines database structure	CREATE, DROP, ALTER, TRUNCATE

DML	Data Manipulation Language	Manipulates data inside tables	INSERT, UPDATE, DELETE
DCL	Data Control Language	Controls access and permissions	GRANT, REVOKE

3. What is Data?

Data refers to raw facts, figures, or information that can be processed to generate meaningful insights. In databases, data is stored in structured formats such as rows and columns.

4. What is a Database?

A database is an organized collection of structured data stored electronically in a computer system. It allows efficient storage, retrieval, and management of data.

5. What is an Index?

An index is a database object used to improve the speed of data retrieval operations. It works like a book index, allowing faster searching of rows in a table without scanning the entire table.

6. What are Joins?

Joins are used to combine rows from two or more tables based on a related column.

Types of Joins:

- INNER JOIN – Returns matching records from both tables
- LEFT JOIN – Returns all records from the left table and matching records from the right table
- RIGHT JOIN – Returns all records from the right table and matching records from the left table
- FULL JOIN – Returns all records when there is a match in either table

7. What is a Datatype? Explain Different Data Types in SQL

A datatype defines the type of data that a column can store.

Common SQL Data Types:

Numeric Types

- INT – Integer values
- FLOAT – Decimal values

- DECIMAL – Fixed precision decimal

Character Types

- CHAR – Fixed-length string
- VARCHAR – Variable-length string
- TEXT – Large text data

Date and Time Types

- DATE – Stores date
- TIME – Stores time
- DATETIME – Stores date and time

Boolean Type

- BOOLEAN – Stores TRUE or FALSE

8. What is an Operator? Explain Different Operators in SQL

Operators are symbols used to perform operations on data.

Types of Operators:

Arithmetic Operators

-
  - (Addition)
-
  - (Subtraction)
-
  - (Multiplication)
- / (Division)

Comparison Operators

- = (Equal)
- (Greater than)
- < (Less than)
- =, <=
- <> or != (Not equal)

Logical Operators

- AND
- OR
- NOT

Special Operators

- BETWEEN
- IN
- LIKE
- IS NULL

9. What is a View?

A view is a virtual table created from a SQL query. It does not store data physically but displays data from one or more tables.

10. Different Commands Available in SQL

SQL commands are categorized as:

- DDL – CREATE, DROP, ALTER, TRUNCATE
- DML – INSERT, UPDATE, DELETE, SELECT
- DCL – GRANT, REVOKE
- TCL (Transaction Control Language) – COMMIT, ROLLBACK, SAVEPOINT

11. What is DBMS?

A Database Management System (DBMS) is software that enables users to create, manage, and manipulate databases. It ensures data security, consistency, integrity, and efficient data handling.

12. What is ER Model?

The Entity-Relationship (ER) Model is a conceptual database design model used to represent entities, attributes, and relationships between them using ER diagrams.

13. How SQL Works – Procedure

1. User writes a SQL query.
2. Query is sent to the database server.
3. Query parser checks syntax.
4. Query optimizer determines the best execution plan.
5. Execution engine processes the query.
6. Result is returned to the user.

14. What is a Table?

A table is a structured format used to store data in rows and columns within a database. Each row represents a record, and each column represents an attribute.

15. Difference Between DROP, TRUNCATE, and DELETE

Command	Purpose	Can Rollback	Removes Structure
DELETE	Removes specific rows	Yes	No
TRUNCATE	Removes all rows	No	No
DROP	Deletes entire table	No	Yes

16. What is an Entity?

An entity is a real-world object or concept that can be identified uniquely in a database (e.g., Student, Employee).

17. What is an Attribute?

An attribute is a property or characteristic of an entity (e.g., StudentID, Name, Age).

18. Strong and Weak Entity

Strong Entity

- Has its own primary key
- Can exist independently

Weak Entity

- Does not have a primary key
- Depends on a strong entity for identification

19. Types of Relationships

- One-to-One (1:1)
- One-to-Many (1:N)
- Many-to-One (N:1)
- Many-to-Many (M:N)

20. How to Create a Database

```
CREATE DATABASE database_name;
```

Example:

```
CREATE DATABASE CompanyDB;
```

21. How to Drop a Database

```
DROP DATABASE database_name;
```

Example:

```
DROP DATABASE CompanyDB;
```

05-03-2026

SQL Assignment – Departments Table Operations

Problem Statement

Create a table named **Departments** with the following columns:

- dept_id
- name
- count_emp_dept
- dept_mail_id

Insert at least 10–15 records into the table.

Perform the following SQL operations on the table:

- CREATE
- INSERT
- SELECT
- WHERE
- ORDER BY
- UPDATE
- DELETE
- ALTER
- TRUNCATE
- DROP
- Logical operators (AND, OR)
- Comparison operators (=, >, <, >=, <=, !=)

1. Create Table

```
CREATE TABLE Departments (
    dept_id INT PRIMARY KEY,
    name VARCHAR(50),
    count_emp_dept INT,
```

```
    dept_mail_id VARCHAR(100)
);
```

2. Insert Records (15 Rows)

```
INSERT INTO Departments VALUES
(1, 'HR', 15, 'hr@company.com'),
(2, 'Finance', 20, 'finance@company.com'),
(3, 'IT', 50, 'it@company.com'),
(4, 'Marketing', 25, 'marketing@company.com'),
(5, 'Sales', 40, 'sales@company.com'),
(6, 'Admin', 10, 'admin@company.com'),
(7, 'Operations', 35, 'operations@company.com'),
(8, 'Legal', 8, 'legal@company.com'),
(9, 'R&D', 30, 'rnd@company.com'),
(10, 'Support', 22, 'support@company.com'),
(11, 'Procurement', 12, 'procurement@company.com'),
(12, 'Logistics', 18, 'logistics@company.com'),
(13, 'Security', 14, 'security@company.com'),
(14, 'Design', 16, 'design@company.com'),
(15, 'QA', 28, 'qa@company.com');
```

3. SELECT Command

Select all records

```
SELECT * FROM Departments;
```

Select specific columns

```
SELECT name, count_emp_dept FROM Departments;
```

4. WHERE Clause with Comparison Operators

Departments with more than 20 employees

```
SELECT * FROM Departments
WHERE count_emp_dept > 20;
```

Departments with exactly 10 employees

```
SELECT * FROM Departments
WHERE count_emp_dept = 10;
```

Departments with employee count not equal to 15

```
SELECT * FROM Departments
WHERE count_emp_dept != 15;
```

5. Logical Operators (AND, OR)

Using AND

```
SELECT * FROM Departments
WHERE count_emp_dept > 20 AND count_emp_dept < 40;
```

Using OR

```
SELECT * FROM Departments
WHERE name = 'HR' OR name = 'Finance';
```

6. ORDER BY

Order by employee count (ascending)

```
SELECT * FROM Departments
ORDER BY count_emp_dept ASC;
```

Order by employee count (descending)

```
SELECT * FROM Departments
ORDER BY count_emp_dept DESC;
```

7. UPDATE Command

Increase employee count of HR department

```
UPDATE Departments
SET count_emp_dept = 18
WHERE dept_id = 1;
```

8. DELETE Command

Delete department with dept_id = 8

```
DELETE FROM Departments
WHERE dept_id = 8;
```

9. ALTER Command

Add a new column (location)

```
ALTER TABLE Departments
ADD location VARCHAR(50);
```

Modify column size

```
ALTER TABLE Departments
MODIFY name VARCHAR(100);
```

10. TRUNCATE Command

Removes all records but keeps table structure.

```
TRUNCATE TABLE Departments;
```

11. DROP Command

Deletes entire table including structure.

```
DROP TABLE Departments;
```

06-03-2026

SQL Constraints Assignment

Task 1

Problem Statement

Create a table named **Products** with the following constraints:

- **product_id** – Primary Key
- **product_name** – Cannot be NULL
- **price** – Must be greater than 0 (CHECK constraint)
- **category** – Default value should be **'accessories'**

Solution

Create Products Table

```
CREATE TABLE Products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100) NOT NULL,
    price DECIMAL(10,2) CHECK (price > 0),
    category VARCHAR(50) DEFAULT 'accessories'
);
```

Insert Sample Records

```
INSERT INTO Products VALUES
(1, 'Mouse', 500, 'electronics'),
(2, 'Keyboard', 800, 'electronics'),
(3, 'USB Cable', 150, DEFAULT),
(4, 'Headphones', 1500, 'audio'),
(5, 'Laptop Stand', 1200, DEFAULT);
```

Task 2

Problem Statement

Create two tables **Department** and **Employee** with a relationship between them.

Department Table

- dept_id – Primary Key
- dept_name

Employee Table

- emp_id – Primary Key
- name
- dept_id – Foreign Key referencing Department table

Solution

Create Department Table

```
CREATE TABLE Department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(100)
);
```

Create Employee Table

```
CREATE TABLE Employee (
    emp_id INT PRIMARY KEY,
    name VARCHAR(100),
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
```

Task 3

Problem Statement

Insert **5 records** into the tables and demonstrate how different constraints generate errors when invalid data is inserted.

Solution

Insert Records into Department

```
INSERT INTO Department VALUES
(1, 'HR'),
(2, 'Finance'),
(3, 'IT'),
(4, 'Marketing'),
(5, 'Sales');
```

Insert Records into Employee

```
INSERT INTO Employee VALUES
(101, 'Rahul', 1),
(102, 'Anita', 2),
(103, 'Kiran', 3),
(104, 'Priya', 4),
(105, 'Arjun', 5);
```

Testing Constraint Errors

1. Primary Key Constraint Error

Trying to insert duplicate **product_id**

```
INSERT INTO Products VALUES
(1, 'Monitor', 5000, 'electronics');
```

Result:
Error occurs because **product_id must be unique**.

2. NOT NULL Constraint Error

```
INSERT INTO Products(product_id, price)
VALUES (6, 1000);
```

Result:
Error occurs because **product_name cannot be NULL**.

3. CHECK Constraint Error

```
INSERT INTO Products VALUES
(7, 'Tablet', -500, 'electronics');
```

Result:
Error occurs because **price must be greater than 0**.

4. DEFAULT Constraint Example

```
INSERT INTO Products(product_id, product_name, price)
VALUES (8, 'Charger', 700);
```

Result:
Category will automatically be stored as **'accessories'**.

5. Foreign Key Constraint Error

Trying to insert employee with a department that does not exist.

```
INSERT INTO Employee VALUES
(106, 'Vikas', 10);
```

Result:
Error occurs because **dept_id 10 does not exist in Department table**.

SQL Joins and Aggregation Assignment

Problem Statement

Create two tables **Employee** and **Departments** with the following structure:

Employee Table

- emp_id
- name
- dept_id
- salary
- address

Departments Table

- dept_id
- dept_name

Insert sample records into both tables and write SQL queries to perform the following operations:

1. Find the employee name and department name
2. Find employees who do not belong to any department
3. Find departments without employees
4. Find employee count per department

5. Find the highest salary in each department
6. Find departments with more than 2 employees
7. Find employees working in the IT department
8. Find employees working in the HR department
9. Find employees working in the Finance department
10. Find employees belonging to the Marketing department
11. Find employees belonging to non-IT departments

Solution

Step 1: Create Tables

```
CREATE TABLE Departments (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50)
);
CREATE TABLE Employee (
    emp_id INT PRIMARY KEY,
    name VARCHAR(50),
    dept_id INT,
    salary INT,
    address VARCHAR(100),
    FOREIGN KEY (dept_id) REFERENCES Departments(dept_id)
);
```

Step 2: Insert Sample Data

Insert Departments

```
INSERT INTO Departments VALUES
(1,'IT'),
(2,'HR'),
(3,'Finance'),
(4,'Marketing'),
(5,'Sales');
```

Insert Employees

```
INSERT INTO Employee VALUES
(101,'Rahul',1,60000,'Bangalore'),
(102,'Anita',2,50000,'Mysore'),
(103,'Kiran',1,70000,'Delhi'),
(104,'Priya',3,55000,'Mumbai'),
(105,'Arjun',4,48000,'Chennai'),
(106,'Sneha',1,65000,'Hyderabad'),
(107,'Vikas',NULL,45000,'Pune');
```

Queries

1. Find Employee Name and Department Name

```
SELECT e.name, d.dept_name
FROM Employee e
INNER JOIN Departments d
ON e.dept_id = d.dept_id;
```

2. Find Employees Who Do Not Belong to Any Department

```
SELECT name
FROM Employee
WHERE dept_id IS NULL;
```

3. Find Departments Without Employees

```
SELECT d.dept_name
FROM Departments d
LEFT JOIN Employee e
ON d.dept_id = e.dept_id
WHERE e.emp_id IS NULL;
```

4. Find Employee Count per Department

```
SELECT dept_id, COUNT(emp_id) AS employee_count
FROM Employee
GROUP BY dept_id;
```

5. Find Highest Salary in Each Department

```
SELECT dept_id, MAX(salary) AS highest_salary
FROM Employee
GROUP BY dept_id;
```

6. Find Departments with More Than 2 Employees

```
SELECT dept_id, COUNT(emp_id) AS employee_count
FROM Employee
GROUP BY dept_id
HAVING COUNT(emp_id) > 2;
```

7. Employees Working in IT Department

```
SELECT e.name
FROM Employee e
JOIN Departments d
ON e.dept_id = d.dept_id
WHERE d.dept_name = 'IT';
```

8. Employees Working in HR Department

```
SELECT e.name
FROM Employee e
JOIN Departments d
ON e.dept_id = d.dept_id
WHERE d.dept_name = 'HR';
```

9. Employees Working in Finance Department

```
SELECT e.name
FROM Employee e
JOIN Departments d
ON e.dept_id = d.dept_id
WHERE d.dept_name = 'Finance';
```

10. Employees Belonging to Marketing Department

```
SELECT e.name
FROM Employee e
JOIN Departments d
ON e.dept_id = d.dept_id
WHERE d.dept_name = 'Marketing';
```

11. Employees Belonging to Non-IT Departments

```
SELECT e.name
FROM Employee e
JOIN Departments d
ON e.dept_id = d.dept_id
WHERE d.dept_name != 'IT';
```

07-03-2026

SQL Aggregation and E-Commerce Data Analysis Assignment

Part 1: Sales Table Aggregation

Problem Statement

Create a table named **Sales** with the following columns:

- **id**
- **product_name**
- **price**

Insert **20–25 records** and perform aggregate operations using the following SQL functions:

- **SUM**
- **MIN**
- **MAX**
- **AVG**
- **COUNT**

Solution

Create Sales Table

```
CREATE TABLE Sales(
    id INT PRIMARY KEY,
    product_name VARCHAR(100),
    price INT
);
```

Insert Sample Data (25 Rows)

```
INSERT INTO Sales VALUES
(1,'Mouse',500),
(2,'Keyboard',800),
(3,'Monitor',7000),
(4,'USB Cable',200),
(5,'Headphones',1500),
(6,'Laptop Stand',1200),
(7,'Webcam',2500),
(8,'Microphone',3000),
(9,'SSD',4500),
(10,'Hard Disk',4000),
(11,'Router',3500),
(12,'Printer',6500),
(13,'Scanner',5000),
(14,'Tablet',15000),
(15,'Smartwatch',9000),
(16,'Speaker',2000),
(17,'Power Bank',1200),
(18,'Charger',700),
(19,'Graphics Card',25000),
(20,'RAM',6000),
(21,'Laptop Bag',1000),
(22,'Cooling Pad',800),
(23,'Pen Drive',600),
(24,'Projector',30000),
(25,'Desk Lamp',900);
```

Aggregate Functions

Total Sales Value

```
SELECT SUM(price) AS total_sales
FROM Sales;
```

Minimum Price

```
SELECT MIN(price) AS minimum_price
FROM Sales;
```

Maximum Price

```
SELECT MAX(price) AS maximum_price
FROM Sales;
```

Average Price

```
SELECT AVG(price) AS average_price
FROM Sales;
```

Total Number of Products

```
SELECT COUNT(*) AS total_products
FROM Sales;
```

Part 2: Amazon E-Commerce Data Analysis

Problem Statement

Create an **Amazon Orders table** to analyze e-commerce sales data.
Perform SQL queries to answer business questions related to:

- Sales performance
- Customer spending
- City-wise analysis
- Category analysis
- Monthly revenue trends

Sample Table Structure

Create Amazon Orders Table

```
CREATE TABLE Amazon_Orders(
    order_id INT PRIMARY KEY,
    customer_name VARCHAR(100),
    city VARCHAR(50),
    product_category VARCHAR(50),
    order_value INT,
    order_date DATE
);
```

Insert Sample Data

```
INSERT INTO Amazon_Orders VALUES
(1,'Rahul','Bangalore','Electronics',15000,'2024-01-10'),
(2,'Anita','Delhi','Clothing',3000,'2024-01-15'),
(3,'Kiran','Mumbai','Electronics',20000,'2024-02-10'),
(4,'Priya','Chennai','Home Appliances',12000,'2024-02-18'),
```

```
(5,'Arjun','Bangalore','Electronics',18000,'2024-03-05'),
(6,'Sneha','Hyderabad','Clothing',2500,'2024-03-15'),
(7,'Vikas','Delhi','Electronics',22000,'2024-03-20'),
(8,'Neha','Mumbai','Books',1200,'2024-04-10'),
(9,'Amit','Bangalore','Electronics',25000,'2024-04-18'),
(10,'Riya','Chennai','Furniture',30000,'2024-04-22'),
(11,'Karthik','Hyderabad','Electronics',17000,'2024-05-10'),
(12,'Meera','Delhi','Clothing',4500,'2024-05-18');
```

SQL Analysis Queries

1. Total Sales of the Company

```
SELECT SUM(order_value) AS total_sales
FROM Amazon_Orders;
```

2. Total Sales by Product Category

```
SELECT product_category, SUM(order_value) AS total_sales
FROM Amazon_Orders
GROUP BY product_category;
```

3. Number of orders per City

```
SELECT city, COUNT(order_id) AS total_orders
FROM Amazon_Orders
GROUP BY city;
```

4. Average Order Value per Customer

```
SELECT customer_name, AVG(order_value) AS avg_order_value
FROM Amazon_Orders
GROUP BY customer_name;
```

5. Highest Spending Customer

```
SELECT customer_name, SUM(order_value) AS total_spent
FROM Amazon_Orders
GROUP BY customer_name
ORDER BY total_spent DESC
LIMIT 1;
```

6. Maximum Order Value in Each Category

```
SELECT product_category, MAX(order_value) AS max_order
FROM Amazon_Orders
GROUP BY product_category;
```

7. Cities with Sales Greater Than 50000

```
SELECT city, SUM(order_value) AS total_sales
FROM Amazon_Orders
GROUP BY city
HAVING SUM(order_value) > 50000;
```

8. Number of Customers in Each City

```
SELECT city, COUNT(DISTINCT customer_name) AS customers
FROM Amazon_Orders
GROUP BY city;
```

9. Category with Most Orders

```
SELECT product_category, COUNT(*) AS total_orders
FROM Amazon_Orders
GROUP BY product_category
ORDER BY total_orders DESC
LIMIT 1;
```

10. Monthly Sales

```
SELECT MONTH(order_date) AS month,
SUM(order_value) AS monthly_sales
FROM Amazon_Orders
GROUP BY MONTH(order_date);
```